

Scala 3で作る数式インタプリタ

2025年07月24日（木）
株式会社ネクストビート

本日の流れ

時間	内容
18:30-18:40	会社紹介
18:40-18:50	イントロ：コンピュータは数式をどう理解する？
18:50-19:00	Step 1: プログラムの"設計図"を作ってみよう
19:00-19:15	ハンズオン（前半）：数式インタプリタを作る
19:15-19:25	休憩
19:25-19:35	Step 2：設計図を動かしてみよう
19:35-19:50	ハンズオン（後半）：言語機能を追加する
19:50-20:00	まとめ・質疑応答

イントロ：プログラミング言語を作るって？

「自分だけのプログラミング言語、作ってみたいくないですか？」

でも...

- 難しそうな理論がたくさん出てきそう...
- コンパイラとか構文解析とか...

今日は難しい部分をスキップして、楽しく作ります！

今日のゴール

1. 数式インタプリタ（電卓）を作る

- `1 + 2` → `3`
- `2 * 3 + 4` → `10`

2. 簡単な言語機能を追加する

- 変数： `a = 10`
- 条件分岐： `if (a > 5) 1 else 0`

90分でできるところまで、一緒に作りましょう！

Step 1: プログラムの"設計図"を作ってみよう

コンピュータは数式をどう理解する？

例： `1 + 2 * 3`

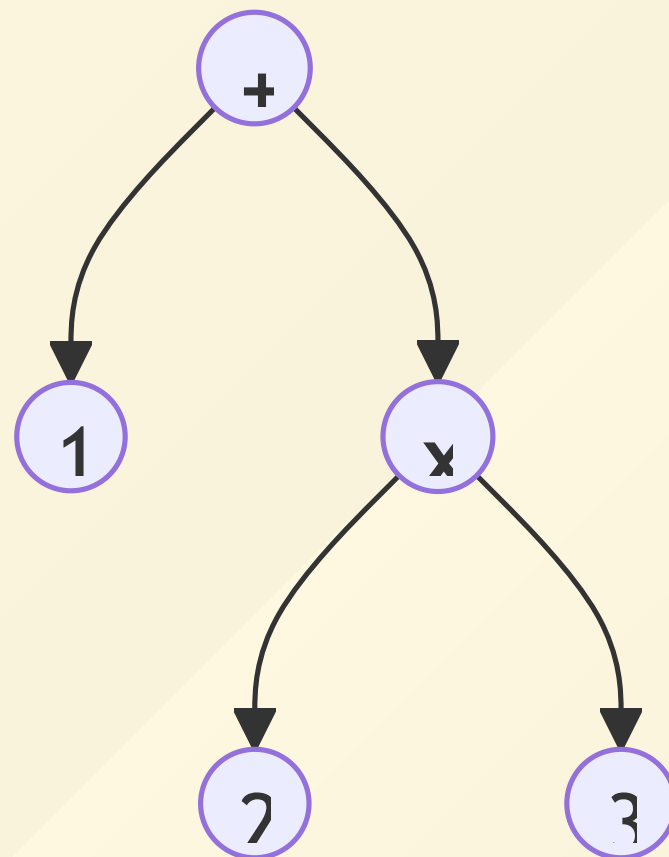
人間の理解：

- 掛け算が優先
- `2 * 3 = 6`
- `1 + 6 = 7`

コンピュータには「優先順位」という概念がない！

抽象構文木 (AST) : プログラムの設計図

`1 + 2 * 3` を「木」の形で表現する



Scala 3で抽象構文木を作る

```
enum Exp {  
  case BinExp(op: String, lhs: Exp, rhs: Exp) // 二項演算  
  case VInt(value: Int)                      // 整数  
}
```

使用例：

```
import Exp.*  
  
VInt(100)                // 100  
BinExp("+", VInt(1), VInt(2)) // 1 + 2  
BinExp("+", VInt(1),  
  BinExp("*", VInt(2), VInt(3))) // 1 + 2 * 3
```


数式の抽象構文

シンプルに定義：

```
式 = 式 + 式    // 加算  
    | 式 - 式    // 減算  
    | 式 * 式    // 乗算  
    | 式 / 式    // 除算  
    | 整数
```

構文解析は今回スキップ！

- 優先順位の処理は複雑
- まずは抽象構文木を直接作ることに集中

ハンズオン（前半）：数式インタプリタを作る

数式インタプリタの実装

再帰的に木をたどって計算するだけ！

```
def eval(exp: Exp): Int = exp match {  
  case VInt(value)      => value  
  case BinExp("+", lhs, rhs) => eval(lhs) + eval(rhs)  
  case BinExp("-", lhs, rhs) => eval(lhs) - eval(rhs)  
  case BinExp("*", lhs, rhs) => eval(lhs) * eval(rhs)  
  case BinExp("/", lhs, rhs) => eval(lhs) / eval(rhs)  
}
```

たったこれだけ！

実行してみよう！

```
// 1 + 2
val exp1 = BinExp("+", VInt(1), VInt(2))
println(eval(exp1)) // 3

// 1 + 2 * 3
val exp2 = BinExp("+",
  VInt(1),
  BinExp("*", VInt(2), VInt(3))
)
println(eval(exp2)) // 7
```

補助関数で読みやすく

```
extension (a: Exp) {  
  def |+|(b: Exp): Exp = BinExp("+", a, b)  
  def |-|(b: Exp): Exp = BinExp("-", a, b)  
  def |*|(b: Exp): Exp = BinExp("*", a, b)  
  def |/|(b: Exp): Exp = BinExp("/", a, b)  
}  
  
def tInt(value: Int): Exp = VInt(value)
```

使用例：

```
val exp = tInt(1) |+| (tInt(2) |*| tInt(3)) // 1 + 2 * 3  
println(eval(exp)) // 7
```

テストを書こう

```
import munit.FunSuite

class EvaluatorSuite extends FunSuite {
  test("1 + 1 == 2") {
    val e = tInt(1) |+| tInt(1)
    assertEquals(eval(e), 2)
  }

  test("2 * 3 + 4 == 10") {
    val e = (tInt(2) |*| tInt(3)) |+| tInt(4)
    assertEquals(eval(e), 10)
  }
}
```

ハンズオンタイム（15分）

やってみよう！

1. Scastie (<https://scastie.scala-lang.org/>) を開く
2. 抽象構文木の定義を書く
3. eval関数を実装する
4. いろいろな式を評価してみる

ヒント：

- `(10 + 20) * 3` はどう表現する？
- `100 / (2 + 3)` は？

休憩（10分）

Step 2：設計図を動かしてみよう

プログラミング言語に必要な機能

今まで：数式だけ

これから追加したい機能：

1. **変数**：値に名前をつける
2. **連接**：複数の式を順番に実行
3. **条件分岐**：条件によって処理を変える

変数機能を追加する

新しい構文木：

```
enum Exp {  
  // 既存  
  case BinExp(op: String, lhs: Exp, rhs: Exp)  
  case VInt(value: Int)  
  
  // 新規追加  
  case Assignment(name: String, expr: Exp) // 代入  
  case Ident(name: String)                // 変数参照  
  case SeqExp(expressions: List[Exp])      // 接続  
}
```

変数を扱うインタプリタ

環境（変数名→値のマップ）を追加：

```
import scala.collection.mutable.{Map => MMap}

def eval(e: Exp, env: MMap[String, Int]): Int = {
  def evalRec(e: Exp): Int = e match {
    case VInt(value) => value
    case BinExp("+", lhs, rhs) => evalRec(lhs) + evalRec(rhs)
    // ... 他の演算

    case Assignment(name, expr) =>
      val v = evalRec(expr)
      env(name) = v // 環境に登録
      v

    case Ident(name) =>
      env.getOrElse(name, sys.error(s"Undefined: $name"))
  }
  evalRec(e)
}
```

連接の実装

複数の式を順番に実行：

```
case SeqExp(bodies) =>
  var result: Int = 0
  bodies.foreach { expr =>
    result = evalRec(expr)
  }
  result // 最後の式の結果を返す
```

使用例：

```
// { a = 10; b = a + 5; b }
val prog = SeqExp(List(
  Assignment("a", VInt(10)),
  Assignment("b", BinExp("+", Ident("a"), VInt(5))),
  Ident("b")
))
eval(prog, MMap.empty) // 15
```

比較演算を追加

```
// 比較演算(結果は0または1)
case BinExp("<", lhs, rhs) =>
  if (evalRec(lhs) < evalRec(rhs)) 1 else 0
case BinExp(">", lhs, rhs) =>
  if (evalRec(lhs) > evalRec(rhs)) 1 else 0
case BinExp("==", lhs, rhs) =>
  if (evalRec(lhs) == evalRec(rhs)) 1 else 0
```

補助関数も追加：

```
extension (a: Exp) {
  def |<|(b: Exp): Exp = BinExp("<", a, b)
  def |>|(b: Exp): Exp = BinExp(">", a, b)
  def |==|(b: Exp): Exp = BinExp("==", a, b)
}
```

条件分岐（if式）を追加

```
enum Exp {  
  // ... 既存の定義  
  case If(condition: Exp, thenClause: Exp, elseClause: Exp)  
}  
  
// evalの実装  
case If(condition, thenClause, elseClause) =>  
  if (evalRec(condition) != 0) // 0以外は真  
    evalRec(thenClause)  
  else  
    evalRec(elseClause)
```

if式の使用例

```
def tIf(cond: Exp, thenE: Exp, elseE: Exp): Exp =  
  If(cond, thenE, elseE)  
  
// if (a > 5) 10 else 20  
val prog = SeqExp(List(  
  Assignment("a", VInt(8)),  
  tIf(  
    Ident("a") |>| VInt(5),  
    VInt(10),  
    VInt(20)  
  )  
))  
eval(prog, MMap.empty) // 10
```


ハンズオン（後半）：言語機能を追加する

練習問題

1. 最大値を求める

```
// a = 10; b = 15; if (a > b) a else b
```

2. 絶対値を求める

```
// x = -5; if (x < 0) -x else x
```

3. 三項演算を模倣

```
// score = 85;  
// grade = if (score >= 90) 5  
//         else if (score >= 80) 4  
//         else 3
```

発展：繰り返し (while式)

```
enum Exp {  
  // ... 既存の定義  
  case While(condition: Exp, body: Exp)  
}  
  
// evalの実装  
case While(condition, body) =>  
  while (evalRec(condition) != 0) {  
    evalRec(body)  
  }  
0 // while式の結果は0
```

while式の使用例

```
// i = 0; sum = 0;
// while (i < 10) {
//   i = i + 1;
//   sum = sum + i;
// }
// sum

val prog = SeqExp(List(
  Assignment("i", VInt(0)),
  Assignment("sum", VInt(0)),
  While(
    Ident("i") |<| VInt(10),
    SeqExp(List(
      Assignment("i", Ident("i") |+| VInt(1)),
      Assignment("sum", Ident("sum") |+| Ident("i"))
    ))
  ),
  Ident("sum")
))
```

ハンズオンタイム（15分）

やってみよう！

1. 変数機能を実装する
2. if式を実装する
3. 練習問題に挑戦する
4. 余裕があればwhile式も！

ヒント：

- 環境は `MMap.empty[String, Int]` で初期化
- 未定義変数の参照はエラーになるよう注意

JSONで構文を表現する（おまけ）

構文解析を避けて、JSONで表現：

```
// 1 + 2 * 3
["+ ", 1, ["*", 2, 3]]

// a = 10; a + 5
["seq",
  ["assign", "a", 10],
  ["+", ["id", "a"], 5]
]
```

これをScalaのASTに変換すれば、テキストでプログラムが書ける！

まとめ

今日作ったもの：

- **数式インタプリタ**（約10行）
- **変数・条件分岐を持つ言語**（約50行）

学んだこと：

1. **抽象構文木**がプログラムの本質
2. **インタプリタ**は再帰的な評価器
3. **構文解析**を避けても言語は作れる

これからの学習

1. 構文解析を学ぶ

- 文字列→ASTの変換
- パーサコンビネータ、PEG

2. 型システムを追加

- 実行前にエラーを検出

3. 最適化やコンパイラ

- より高速な実行

自分だけのプログラミング言語を作ってみよう！

質疑応答

ご質問はありますか？

参考資料

- **Scala 3公式サイト**: <https://scala-lang.org/>
- **本日のコード**: GitHub（後日公開）
- **書籍**:
 - 「Go言語で作るインタプリタ」
 - 「WEB+DB PRESS Vol.125」

懇親会のご案内

20:00から9Fにて開催します。ぜひご参加ください！

ありがとうございました！

株式会社ネクストビート

エンジニア採用も積極的に行っています

HP: <https://www.nextbeat.co.jp/>

Nextbeat Tech Bar

定期的に技術イベントを開催中！